CSE-515 End Semester Report

IoT-Based Model in Smart Urban Traffic Control: Graph Theory and Genetic Algorithm



Group 9

Aniket Johri (20124008)

Gauri D (20124020)

Krishan Malhotra (20124027)

Motivation

Urban traffic congestion is a pressing issue, particularly in cities with dense populations and limited infrastructure like Varanasi, where narrow roads, mixed traffic, and the rapid increase in vehicles exacerbate traffic delays. Traditional traffic control methods often fail to respond efficiently to the dynamic nature of traffic flows, especially during peak hours or special events like religious festivals in cities such as Varanasi, which draw large crowds. The environmental impact, with increasing air pollution from idling vehicles, further highlights the need for an intelligent solution.

Key features and benefits

By leveraging the Internet of Things (IoT), graph theory, and genetic algorithms, we can develop a more responsive traffic management system. IoT devices collect real-time traffic data, enabling quick adaptation to changing conditions.

Graph theory is used to model urban roads, providing a structured representation of the traffic network.

Genetic algorithms optimize traffic flow by finding the best possible paths for vehicles based on real-time data.

The system aims to:

- Minimize traffic jams.
- Reduce vehicle waiting times.
- Lower environmental pollution caused by idling vehicles.

This approach is highly relevant in cities like Varanasi, where infrastructure constraints and increasing traffic demand require efficient traffic management solutions.

Related works

Several works have addressed the issue of traffic management using various techniques:

- **IoT for Traffic Data Collection**: IoT has been used for real-time traffic data collection, enabling better monitoring and decision-making in traffic systems.
- **Graph Theory in Traffic Control**: Many studies have modeled urban road networks as graphs, where intersections are vertices and roads are edges. These studies focus on traffic optimization by ensuring connectivity between nodes and identifying optimal routes
- **Genetic Algorithms**: Genetic algorithms have been employed in optimization problems, including traffic management, where they are used to find the best solutions for reducing congestion and improving the efficiency of traffic flow

References

- P. Kuppusamy, R. Kalpana, P. V. V. Rao, ”Optimized traffic control and data processing using IoT,” Clust. Comput., 22: 2169-2178, 2019.
- H. Tahaei, F. Afifi, A. Asemi, F. Zaki, N. B. Anuar, ”The rise of traffic classification in IoT networks: A survey,” J. Netw. Comput. Appl. 154, 2020.
- S. Geetha, D. Cicilia, ”IoT enabled intelligent bus transportation system,” in 2nd International Conference on Communication and Electronics Systems (ICCES), pp. 7-11, IEEE, 2017.
- Y. Huang, L. Wang, Y. Hou, W. Zhang, Y. Zhang, ”A prototype IoT based wireless sensor network for traffic information monitoring,” International Journal of Pavement Research and Technology, 11(2): 146-152, 2018.
- J. Guerrero-Ibanez, S. Zeadally, J. Contreras-Castillo, ”Sensor technologies for intelligent transportation systems,” Sensors 18(4), 2018.

However, while these approaches have demonstrated potential, they often fail to combine IoT's real-time data collection with graph theory and genetic algorithms in a single system that dynamically adapts to changing traffic conditions.

Problem Statement

The increase in urban population and vehicles has led to severe traffic congestion, especially during peak hours or special events. Traditional traffic systems with fixed signal timings cannot efficiently manage dynamic traffic flows. The primary goal is to reduce congestion by collecting real-time traffic data using IoT devices, modeling urban roads as graphs, and applying genetic algorithms to optimize traffic flow and minimize waiting times and queue lengths.

The architecture of the proposed system can be visualized as a three-layered model:

1. **IoT Layer**: Collects real-time data from sensors installed at intersections.
2. **Graph Theory Layer**: Models the road network where vertices represent intersections and edges represent roads.
3. **Genetic Algorithm Layer**: Optimizes the path by reducing congestion based on real-time data.

Mathematical Equations

Vertices and Graph Model: The intersections in the road network are represented as vertices ($v_1, v_2, \ldots, v_n$) in a graph. The roads connecting these intersections are the edges between the vertices. The aim is to orient these edges (i.e., make them one-way or two-way) so that the graph becomes strongly connected. This means that every intersection has both an entry and exit route, allowing for bidirectional travel between any two intersections.

Weights on the Graph: To model traffic flow, each road has an associated weight, which reflects the traffic intensity on that road:

- For each intersection v_i, there are several weights $w_{i,1}, w_{i,2}, \ldots, w_{i,k}$, representing the traffic on the roads leading to that intersection. These weights are gathered using IoT devices, such as sensors placed at the intersections that collect real-time traffic data.
- Additionally, the **distance between intersections** $d_{i,j}$ is considered as part of the weight. This is the physical distance between two intersections v_i and v_j, typically measured using Euclidean distance.

Edge Weights Between Intersections: The total weight of a road (edge) between two intersections v_i and v_j is calculated by multiplying the output traffic weight from v_i, the input traffic weight to v_j, and the distance between them. The formula for the weight of the edge from v_i to v_j is:

$$W_{i,j} = w_{i,l} \times w_{j,l} \times d_{i,j}$$

where:

- $w_{i,l}$ is the output weight from v_i to v_j,
- $w_{j,l}$ is the input weight to v_j,
- $d_{i,j}$ is the Euclidean distance between intersections v_i and v_j.

Waiting Time Formula:

We'll calculate waiting time W_i at each intersection i using the provided formula:

$$W_i = \frac{\sum_{j=1}^{k} w_{i,j} \cdot d_{i,j}}{\text{signal cycle time}}$$

where:

- $w_{i,j}$: traffic weight for road j at intersection i,
- $d_{i,j}$: distance from the previous intersection to i,
- `signal_cycle_time`: duration of a full traffic light cycle in seconds.

Using which we see that the waiting time reduces as we optimize our code.

Algorithm to Orient the Context Graph

- **Initial Step**
  Start with one vertex, assign it a number, and select an adjacent vertex with the minimum edge weight to connect. Assign numbers to both vertices and direct the edge between them.
- **General Case**
  For each numbered vertex, select an unnumbered adjacent vertex with the minimum weight and continue the process. If no unnumbered vertices remain, return to the previous vertex and repeat the process.
- **Directed Spanning Subtree**
  This method generates a directed sub-tree that spans all vertices, ensuring a connected graph. The direction of remaining edges is assigned based on the orientation of the sub-tree.
- **Strongly Connected Graph**
  The result is a strongly connected graph with directed edges.

Difference between our work and existing work

Existing Framework:

The current urban traffic network management leverages graph theory to model roads as directed graphs, where intersections are represented as vertices and roads as edges. The primary goal is to achieve a strongly-connected graph that optimizes vehicle flow across intersections. This is done by ensuring strong connectivity, biconnectivity, and optimal weighting of edges based on real-time traffic data collected through IoT devices.

Proposed Improvement:

We can incorporate a few key improvements that will enhance its performance and solution quality, especially for larger graphs and more complex traffic networks.

Here’s the optimization proposal:

1. Use of Parallelism for Fitness Calculation

- **Optimization**: The fitness calculation for each chromosome can be computationally expensive, especially with large populations or complex graphs. By using parallel processing (e.g., Python's `concurrent.futures` or multiprocessing), we can evaluate

fitness for multiple chromosomes simultaneously, significantly speeding up the algorithm.

- **Justification**: Traffic networks often contain many intersections and connections, so calculating the fitness in parallel can reduce runtime, making the algorithm more scalable and efficient for larger graphs.

2. Adaptive Mutation Rate

- **Optimization**: Implement an adaptive mutation rate that decreases as the algorithm progresses. Start with a higher mutation rate to explore diverse solutions, then gradually reduce it to refine and optimize the best solutions.
- **Justification**: Early generations benefit from high mutation, which prevents premature convergence and allows the algorithm to explore different orientations. As the algorithm reaches later generations, a lower mutation rate helps stabilize the solution, focusing on fine-tuning rather than exploration.

3. Hybrid Selection with Tournament Selection

- **Optimization**: Instead of relying solely on the fitness-sorted selection, implement **tournament selection** where a subset of chromosomes is chosen randomly, and the best within that subset is selected. This approach balances exploration and exploitation.
- **Justification**: Tournament selection helps maintain diversity by reducing the chance of selecting only the best chromosomes in every generation, which can lead to a more diverse set of offspring and prevent premature convergence.

While the existing algorithm effectively orients edges based on minimum weights (traffic load and distance), we aim to enhance this solution using genetic algorithms (GA). Genetic algorithms are a type of evolutionary algorithm inspired by natural selection, ideal for optimization problems where multiple solutions are possible. In this context, they can optimize the orientation of edges to further minimize travel times, balance traffic loads, and adapt dynamically to changing conditions.

Expected Results:

Incorporating a genetic algorithm on top of the existing graph-based solution will:

- Provide a more robust and flexible approach to traffic management.
- Yield better solutions for edge orientation that account for both current and predicted traffic conditions.
- Adapt to unforeseen changes in real-time, further enhancing urban mobility and reducing congestion.

Genetic Algorithm for Optimization

The Genetic Algorithm (GA) is used to optimize the traffic flow and road orientation in the urban network. The algorithm operates through several phases:

Chromosome Representation

Each chromosome represents a potential solution to the traffic flow problem (strongly correlated graph), encoding a specific orientation of the directed graph, where road directions (one-way or two-way) are assigned to minimize congestion.

Fitness Function

A fitness function evaluates the efficiency of each chromosome by calculating the total travel cost (sum of traffic loads and distances) between intersections. The goal is to minimize this travel cost.

- Fitness Function: The total path cost F_r is calculated as:

$$F_r = \sum_{i,j \in V(G)} W_{i,j} + \min(W_{j,i})$$

where $W_{i,j}$ is the weight of the path from i to j, and $W_{j,i}$ is the reverse path weight.

Genetic Operators

Selection: High-performing chromosomes are selected for reproduction.

Crossover: Pairs of chromosomes are combined to create new solutions, ensuring the most promising traits are passed onto the next generation.

Mutation: A small percentage of chromosomes undergo random changes to explore new possibilities and avoid local optima.

Optimization Process

The GA repeats the process of selection, crossover, and mutation over several generations. As the algorithm progresses, the solutions converge toward an optimal traffic flow configuration, minimizing both waiting times and congestion.

Pseudo-Code

Initialize population of graphs G

For each generation:

Calculate fitness function for each graph in population

Sort graphs by fitness score

Select top-performing graphs for crossover

Perform crossover and mutation to create new population

Replace least-fit graphs with new offspring

Evaluate new population

If previous generations return same answer, return graph

End for

Code

Python

```python
import random
import networkx as nx
from concurrent.futures import ThreadPoolExecutor

# Function to create an initial traffic graph
def create_traffic_graph(num_intersections):
    graph = nx.DiGraph()
    for i in range(num_intersections):
        for j in range(i + 1, num_intersections):
            weight = random.randint(1, 10)  # Simulate distance/traffic cost
            graph.add_edge(i, j, weight=weight)
            graph.add_edge(j, i, weight=weight)
    return graph

# Fitness function: Calculates the path cost of a graph configuration
def fitness(graph):
    total_cost = 0
    for u in graph.nodes():
        for v in graph.nodes():
            if u != v:  # Ensure we don't include self-loops
                if graph.has_edge(u, v):
                    forward_cost = graph[u][v]['weight']
                else:
                    forward_cost = float('inf')  # No direct path

                if graph.has_edge(v, u):
                    reverse_cost = graph[v][u]['weight']
                else:
                    reverse_cost = float('inf')  # No reverse path

                total_cost += forward_cost + min(forward_cost, reverse_cost)
    return total_cost

# Parallelized fitness calculation
def calculate_fitness_population(population):
    with ThreadPoolExecutor() as executor:
        fitness_scores = list(executor.map(fitness, population))
    return fitness_scores

# Initialize a population of random graph configurations
def initialize_population(graph, population_size):
    population = []
```

```
    for _ in range(population_size):
        chromosome = graph.copy()
        for u, v in chromosome.edges():
            if random.random() > 0.5:
                chromosome[u][v]['weight'], chromosome[v][u]['weight'] =
chromosome[v][u]['weight'], chromosome[u][v]['weight']
        population.append(chromosome)
    return population

# Selection: Hybrid tournament selection
def selection(population, fitness_scores, num_select=2):
    selected = []
    for _ in range(num_select):
        tournament = random.sample(list(zip(population, fitness_scores)), k=5)
        selected.append(min(tournament, key=lambda x: x[1])[0])  # Choose the best
in the tournament
    return selected

# Crossover: Mix edges between two parents to create a child chromosome
def crossover(parent1, parent2):
    child = parent1.copy()
    for u, v in parent1.edges():
        if random.random() > 0.5:
            child[u][v]['weight'] = parent2[u][v]['weight']
    return child

# Mutation with adaptive mutation rate
def mutate(chromosome, mutation_rate):
    for u, v in chromosome.edges():
        if random.random() < mutation_rate:
            chromosome[u][v]['weight'], chromosome[v][u]['weight'] =
chromosome[v][u]['weight'], chromosome[u][v]['weight']
    return chromosome

# Calculate average waiting time based on total path cost in the graph
def calculate_average_waiting_time(graph):
    total_weight = sum(fitness(graph) for u in graph.nodes() for v in graph.nodes()
if u != v)
    average_waiting_time = total_weight / len(graph.nodes())
    return average_waiting_time

# Genetic Algorithm with adaptive mutation rate and elitism
def genetic_algorithm(graph, population_size=10, generations=20,
crossover_rate=0.8, initial_mutation_rate=0.2):
    population = initialize_population(graph, population_size)
    best_solution = min(population, key=fitness)
    best_fitness = fitness(best_solution)
```

```
    for generation in range(generations):
        # Adaptive mutation rate
        mutation_rate = initial_mutation_rate * (1 - (generation / generations))

        # Calculate fitness for each chromosome in parallel
        fitness_scores = calculate_fitness_population(population)

        # Update best solution found so far
        current_best = population[fitness_scores.index(min(fitness_scores))]
        current_best_fitness = min(fitness_scores)
        if current_best_fitness < best_fitness:
            best_solution = current_best
            best_fitness = current_best_fitness

        print(f"Generation {generation + 1}, Best Path Cost: {best_fitness}")

        # Selection of parents using tournament selection
        num_parents = int(population_size * crossover_rate)
        parents = selection(population, fitness_scores, num_parents)

        # Crossover and mutation to produce next generation
        children = []
        while len(children) < population_size - num_parents:  # Elite retention
            parent1, parent2 = random.sample(parents, 2)
            child = crossover(parent1, parent2)
            child = mutate(child, mutation_rate)
            children.append(child)

        # Elite retention
        elites = [population[i] for i in sorted(range(len(fitness_scores)),
key=lambda x: fitness_scores[x])[:num_parents]]
        population = elites + children

    return best_solution

# Set up parameters and run the genetic algorithm
num_intersections = 5  # Number of intersections in the traffic network
traffic_graph = create_traffic_graph(num_intersections)

# Calculate initial average waiting time before optimization
initial_waiting_time = calculate_average_waiting_time(traffic_graph)
print("Initial Average Waiting Time:", initial_waiting_time)

# Run the genetic algorithm to find the best graph configuration
best_graph = genetic_algorithm(traffic_graph)
```

```python
# Calculate average waiting time after optimization
optimized_waiting_time = calculate_average_waiting_time(best_graph)
print("Optimized Average Waiting Time:", optimized_waiting_time)

print("Waiting Time Reduction:", initial_waiting_time - optimized_waiting_time)
```

Result:

